

Laboratorio 3

SQL – Query

rif. paragrafi 6.1, 6.2, 6.3.6 – 6.3.8 libro

Corso di Basi di Dati

A.A. 2012/2013

Presented by
Matteo Lissandrini
matteo.lissandrini@gmail.com

Courtesy of:
Francesco Corcoglioniti
corcoglioniti@disi.unitn.it

PostgreSQL

- PostgreSQL (a.k.a. Postgres) è un DBMS relazionale open source disponibile per Windows, Linux e MacOS X
 - <http://www.postgresql.org/>
- PostgreSQL gestisce database
 - un database è composto da uno o più schemi
 - ogni schema può contenere zero o più tabelle (e altri 'oggetti' come viste, trigger, stored procedures, ...)
- PostgreSQL si usa
 - sui PC in laboratorio sotto **Linux**, per gli esercizi su SQL
 - sul proprio PC (consigliato, non obbligatorio)
 - per esercitarsi a casa
 - per usare il proprio portatile in aula installando in locale il server PostgreSQL, in quanto da portatile non è possibile ad accedere al server in aula

PostgreSQL – Server e client

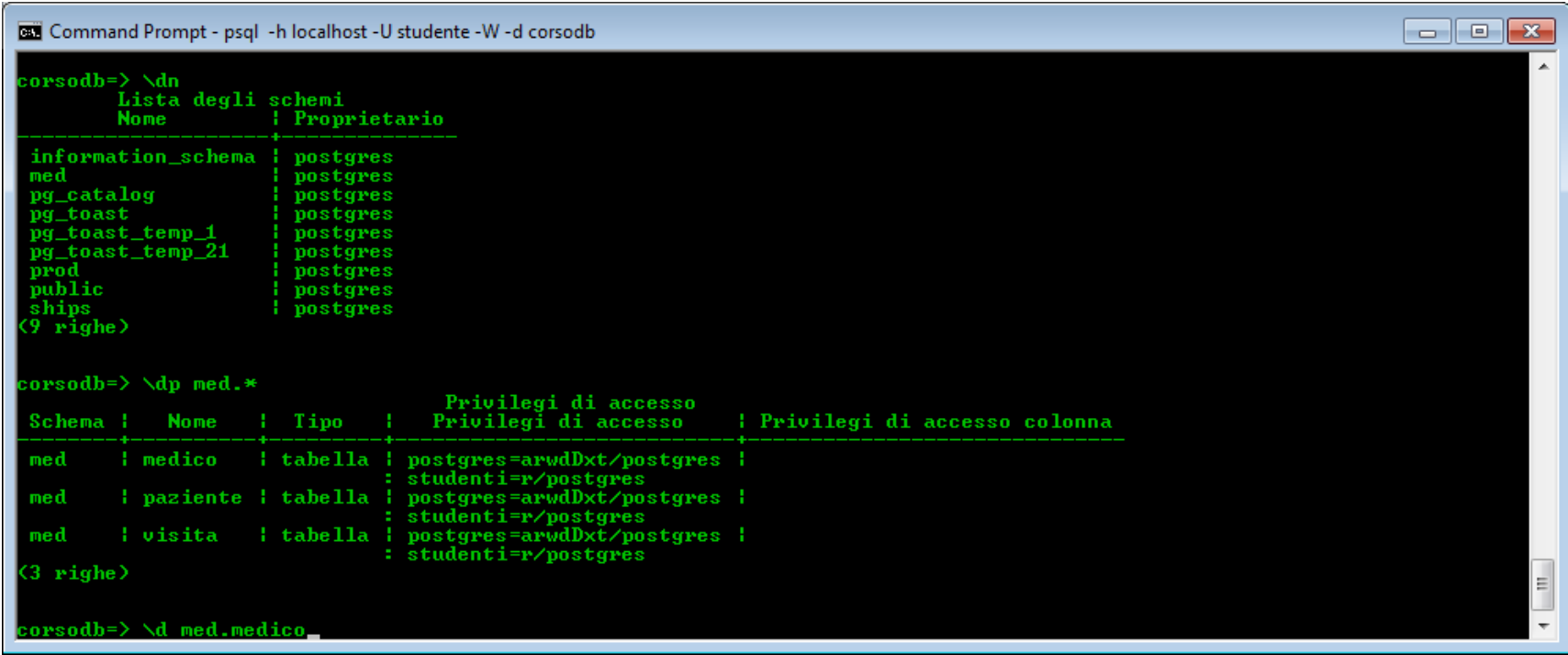
- PostgreSQL è un sistema client/server
 - il server gestisce i database (memorizzati su disco in locale) e accetta connessioni di rete dai client
 - il client si connettono al server via rete e permettono all'utente o ad una applicazione di interagire con esso e manipolare i database mediante comandi SQL o proprietari di PostgreSQL
- Tra i client vi sono
 - psql – client a riga di comando
 - pgAdmin III – client a interfaccia grafica, più comodo
 - librerie client (e.g. JDBC) che permettono di accedere ad un server PostgreSQL da altre applicazioni

PostgreSQL – Installazione (1/2)

- Per Windows XP / Vista / Seven
 - usare il “one click installer” della versione più recente di PostgreSQL
 - <http://www.enterprisedb.com/products-services-training/pgdownload#windows>
 - scegliere Win x86-32 o Win x86-64 a seconda che Windows sia a 32 o 64 bit
 - durante l’installazione, scegliere e ricordarsi la password di ‘postgres’
 - non eseguire Application Stack Builder
- Per Linux Ubuntu / Debian, eseguire da root i comandi
 - **sudo apt-get install postgresql-9.1**
 - **sudo apt-get install pgadmin3**
 - **sudo -u postgres psql postgres**
 - in psql, eseguire **\password postgres**, scegliere la password di admin e uscire con **\q**
 - In caso di problemi: <http://russbrooks.com/2010/12/4/install-postgresql-9-on-ubuntu-linux>
- Per Mac OS X
 - usare il “one click installer”
 - in caso di errore su Mac OS X “Lion” 10.7.1, riavviare il PC e riprovare
 - <http://forums.enterprisedb.com/posts/list/2876.page>
- Altre info: http://wiki.postgresql.org/wiki/Detailed_installation_guides

PostgreSQL – psql (1/2)

- Per connettersi
 - aprire un terminale
 - e.g. on Windows: Applications / Accessories / Terminal
 - eseguire **psql -h *hostname* -U *username* -W -d *database***
 - La password viene chiesta in seguito



```
cmd Command Prompt - psql -h localhost -U studente -W -d corsodb

corsodb=> \dn
      Lista degli schemi
      Nome               | Proprietario
-----+-----
information_schema      | postgres
med                      | postgres
pg_catalog               | postgres
pg_toast                 | postgres
pg_toast_temp_1         | postgres
pg_toast_temp_21        | postgres
prod                     | postgres
public                   | postgres
ships                    | postgres
<9 righe>

corsodb=> \dp med.*
Schema | Nome      | Tipo  | Privilegi di accesso
-----+-----+-----+-----
med    | medico   | tabella | postgres=arwdDxt/postgres |
      |          |         | studenti=r/postgres       |
med    | paziente | tabella | postgres=arwdDxt/postgres |
      |          |         | studenti=r/postgres       |
med    | visita  | tabella | postgres=arwdDxt/postgres |
      |          |         | studenti=r/postgres       |
<3 righe>

corsodb=> \d med.medico_
```

PostgreSQL – psql (2/2)

- Il client psql accetta due tipi di comandi
 - comandi SQL (select..., insert..., ...)
 - comandi specifici di psql
- Comandi SQL:
 - si possono scrivere su più linee
 - vanno terminati dal carattere “;”
- Comandi specifici psql:
 - **\dn** – mostra gli schemi disponibili
 - **\dp schema.*** – mostra le tabelle in uno schema
 - **\d schema.tabella** – mostra le colonne di una tabella
 - **\h** – help
 - **\q** – termina

PostgreSQL – pgAdmin III (1/3)

The screenshot shows the pgAdmin III interface. In the 'Esploratore degli oggetti' (Object Explorer) on the left, the 'laptop' table is selected under the 'prod' schema. A red circle highlights the 'laptop' table, and a red arrow points from it to the 'Query' window. The 'Query' window, titled 'Query - corsodb su postgres@localhost:5432 *', shows the SQL query: `SELECT * FROM prod.product;`. Below the query, the 'Riquadro di «output»' (Output Panel) displays the results of the query. The output is a table with 4 rows and 4 columns: 'maker', 'model', 'type', and 'speed'. The first 4 rows are highlighted in blue. The status bar at the bottom of the output panel indicates '27 rows, 72 ms'.

	maker character(1)	model integer	type character varying(9)
1	A	1001	pc
2	A	1002	pc
3	A	1003	pc
4	A	2004	laptop

OK. Unix Ln 1 col 9 car 9 27 rows. 72 ms

PostgreSQL – pgAdmin III (2/3)

- Per connettersi
 - eseguire
Programming / pgAdmin III
 - selezionare
File / Aggiungi server...
 - compilare la form
 - nome arbitrario
 - Hostname: indirizzo del server
 - utente: username **studente**
 - password: **password account**

Registrazione di un nuovo server

Proprietà

Nome: server postgres laboratorio

Host: 192.168.162.91

Porta: 5432

SSL: [dropdown]

DB di gestione: postgres

Nome utente: studente

Password: [masked]

Ricorda password: ☒

Ripristino ambiente?: ☒

Restrizioni del database: [empty text box]

Servizio: [empty text box]

Connetti adesso: ☒

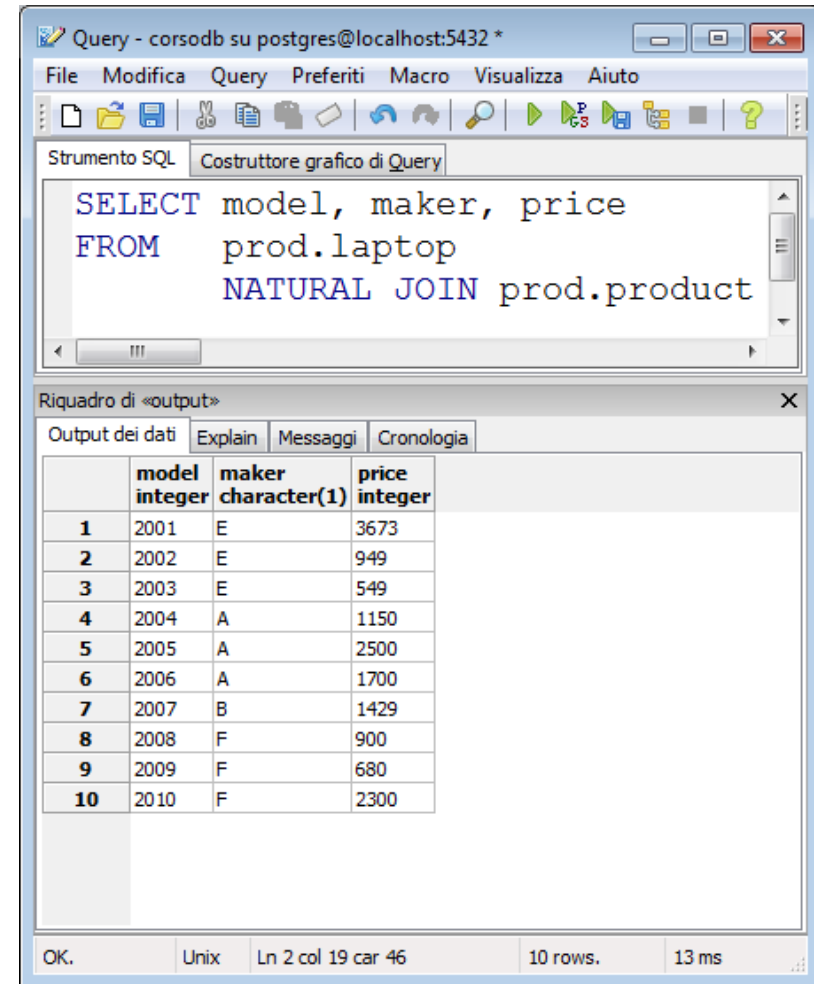
Colore: [empty text box] [color picker]

Guida OK Annulla

PostgreSQL – pgAdmin III (3/3)

Una volta connessi, si può:

- navigare schemi e tabelle nel DB, usando il pannello sx di pgAdmin
- visualizzare gli attributi di una tabella
 - click dx su tabella, quindi “Proprietà...”
- visualizzare i dati in una tabella
 - click dx su tabella, quindi Visualizza dati / Mostra tutte le righe
- eseguire comandi arbitrari SQL sul database
 - selezionare il pulsante SQL (sesto pulsante nella toolbar) per aprire il pannello SQL (vedi figura)



PostgreSQL – Post-Installazione

- Una volta installato PostgreSQL e i suoi client (psql e pgAdmin III) si può importare il database usato in laboratorio
- Scaricare i File SQL da <http://j.mp/BasiDatiSchemas>
- Usando pgAdmin III
 - connettersi al server installato sul proprio PC
 - creare un nuovo database
 - click dx su Database, “Nuovo database...”,
 - immettere il nome del database e selezionare OK
 - selezionare il nuovo database e aprire il pannello SQL
 - per ogni file **create_xxx_db.sql** nello ZIP
 - aprire il file con un editor
 - copiare il contenuto del file nel pannello SQL, rimpiazzando eventuale codice SQL presente nel pannello
 - eseguire – ciascun script crea e popola uno schema con relative tabelle

Esercizio 1 – Prodotti

rif. esercizi 6.1.3 (query 1-6), 6.2.2 (query 7-12)

Sia dato lo schema:

- product (model, maker, type)
- pc (model, speed, ram, hd, price)
- laptop (model, speed, ram, hd, screen, price)
- printer (model, color, type, price)

Esprimere in SQL le seguenti query:

1. Trovare modello, memoria e velocità per laptop di almeno 1200\$
2. Elencare le stampanti a colori (tornare tutti i dati a disposizione)
3. Elencare modello e HD dei PC di almeno 3.0 GHz e con prezzo inferiore a 1000\$
4. Elencare modello, velocità e dimensione HD per i PC con prezzo minore di 800\$
5. Come nella query (1), ma ridenominando *ram* in *gigabytes* e *speed* in *gigahertz*
6. Ritornare i produttori di laptop
7. Elencare i produttori che vendono PC ma non laptop
8. Trovare produttore e velocità dei laptop con HD di almeno 100 GB
9. Trovare modello e prezzo di tutti i prodotti venduti dal produttore C
10. Trovare le coppie di PC con stessa RAM e HD (evitare ripetizioni)
11. Trovare le velocità di CPU in comune a due o più PC
12. Trovare i produttori di almeno due computer (PC o laptop) veloci almeno 2 GHz

Esercizio 1 – Soluzioni (1/6)

1. Trovare modello, memoria e dimensione schermo per laptop di almeno 1200\$

```
SELECT model, ram, screen
FROM prod.laptop
WHERE price >= 1200;
```

2. Elencare le stampanti a colori (tornare tutti i dati a disposizione)

```
SELECT *
FROM prod.printer
WHERE color;
```

3. Elencare modello e HD dei PC di almeno 3.0 GHz e con prezzo inferiore a 1000\$

```
SELECT model, hd
FROM prod.pc
WHERE speed >= 3.0 AND price < 1000;
```

4. Elencare modello, velocità e dimensione HD per i PC con prezzo minore di 800\$

```
SELECT model, speed, hd
FROM prod.pc
WHERE price < 800;
```

Esercizio 1 – Soluzioni (2/6)

5. Come nella query (1), ma ridenominando ram in gigabytes e speed in gigahertz

```
SELECT model, ram AS gigabytes, speed AS gigahertz  
FROM prod.laptop  
WHERE price >= 1200;
```

6. Ritornare i produttori di laptop

```
SELECT DISTINCT maker  
FROM prod.product  
WHERE type = 'laptop';
```

7. Elencare i produttori che vendono PC ma non laptop

```
( SELECT maker  
  FROM prod.product  
  WHERE type = 'pc' )  
EXCEPT  
( SELECT maker  
  FROM prod.product  
  WHERE type = 'laptop' );
```

Esercizio 1 – Soluzioni (3/6)

8. Trovare produttore e velocità dei laptop con HD di almeno 100 GB

```
SELECT maker, speed  
FROM prod.product p, prod.laptop l  
WHERE hd >= 100 AND p.model = l.model;
```

oppure:

```
SELECT maker, speed  
FROM prod.product NATURAL JOIN prod.laptop  
WHERE hd >= 100;
```

oppure:

```
SELECT maker, speed  
FROM prod.product p JOIN prod.laptop l ON p.model = l.model  
WHERE hd >= 100;
```

Esercizio 1 – Soluzioni (4/6)

9. Trovare modello e prezzo di tutti i prodotti venduti dal produttore C

```
( SELECT model, price
  FROM prod.pc NATURAL JOIN prod.product
  WHERE maker = 'C' )
UNION
( SELECT model, price
  FROM prod.laptop NATURAL JOIN prod.product
  WHERE maker = 'C' )
UNION
( SELECT model, price
  FROM prod.printer NATURAL JOIN prod.product
  WHERE maker = 'C' )
```

10. Trovare le coppie di PC con stessa RAM e HD (evitare ripetizioni)

```
SELECT pc1.model AS model1, pc2.model AS model2
FROM prod.pc pc1, prod.pc pc2
WHERE pc1.ram = pc2.ram AND pc1.hd = pc2.hd AND pc1.model <
pc2.model;
```

Esercizio 1 – Soluzioni (5/6)

11. Trovare le velocità di CPU in comune a due o più PC

```
SELECT DISTINCT pc1.speed AS speed
FROM prod.pc pc1, prod.pc pc2
WHERE pc1.model <> pc2.model AND pc1.speed = pc2.speed;
```

12. Trovare i produttori di almeno due computer (PC o laptop) veloci almeno 2 GHz

```
( SELECT p1.make
  FROM prod.product p1, prod.product p2,
       prod.pc pc1, prod.pc pc2
 WHERE p1.make = p2.make AND p1.model = pc1.model
       AND p2.model = pc2.model and p1.model <> p2.model
       AND pc1.speed >= 2.0 AND pc2.speed >= 2.0 )
UNION
( SELECT p1.make
  FROM prod.product p1, prod.product p2,
       prod.pc pc1, prod.laptop lp2
 WHERE p1.make = p2.make AND p1.model = pc1.model
       AND p2.model = lp2.model and p1.model <> lp2.model
       AND pc1.speed >= 2.0 AND lp2.speed >= 2.0 )
```

[continua]

Esercizio 1 – Soluzioni (6/6)

12. Trovare i produttori di almeno due computer (PC o laptop) veloci almeno 2 GHz

[continua]

```
UNION
( SELECT p1.make
  FROM prod.product p1, prod.product p2,
      prod.laptop lp1, prod.pc pc2
 WHERE p1.make = p2.make AND p1.model = lp1.model
      AND p2.model = pc2.model and p1.model <> p2.model
      AND lp1.speed >= 2.0 AND pc2.speed >= 2.0 )
UNION
( SELECT p1.make
  FROM prod.product p1, prod.product p2,
      prod.laptop lp1, prod.laptop lp2
 WHERE p1.make = p2.make AND p1.model = lp1.model
      AND p2.model = lp2.model and p1.model <> p2.model
      AND lp1.speed >= 2.0 AND lp2.speed >= 2.0 );
```

Esercizio 2 – Navi da battaglia

rif. esercizi 6.1.4 (query 1-6), 6.2.3 (query 7-12)

Sia dato lo schema:

- classes (class, type, country, num_guns, bore, displacement)
- ships (name, class, launched)
- outcomes (ship, battle, result)
- battles (name, date)

Esprimere in SQL le seguenti query:

1. Elencare nome classe e paese di tutte le classi con almeno 12 cannoni
2. Elencare i nomi delle navi varate prima del 1915, cambiando *name* in *shipName*
3. Elencare i nomi delle navi che iniziano con 'M'
4. Elencare gli affondamenti in battaglia (nome nave e nome battaglia)
5. Elencare le navi che hanno lo stesso nome della rispettiva classe
6. Elencare i nomi delle navi il cui nome è composto da almeno tre parole
7. Elencare nome, stazza e numero cannoni delle navi della battaglia di Guadalcanal
8. Elencare le navi con stazza maggiore di 37000 t
9. Elencare le navi presenti nel DB (attenzione a navi in Outcomes non in Ships)
10. Trovare le battaglie con almeno due navi dello stesso paese
11. Trovare i paesi che avevano sia corazzate (bb) che incrociatori (bc)

Esercizio 2 – Soluzioni (1/4)

1. Elencare nome classe e paese di tutte le classi con almeno 12 cannoni

```
SELECT class, country  
FROM ships.classes  
WHERE num_guns >= 12;
```

2. Elencare i nomi delle navi varate prima del 1915, cambiando name in shipName

```
SELECT name AS "shipName"  
FROM ships.ships  
WHERE launched < 1915;
```

3. Elencare i nomi delle navi che iniziano con 'M'

```
SELECT name  
FROM ships.ships  
WHERE name LIKE 'M%';
```

4. Elencare gli affondamenti in battaglia (nome nave e nome battaglia)

```
SELECT ship, battle  
FROM ships.outcomes  
WHERE result = 'sunk';
```

Esercizio 2 – Soluzioni (2/4)

5. Elencare le navi che hanno lo stesso nome della rispettiva classe

```
SELECT name
FROM ships.ships
WHERE name = class;
```

6. Elencare i nomi delle navi il cui nome è composto da almeno tre parole

```
( SELECT name
  FROM ships.ships
  WHERE name LIKE '% % %' )
UNION
( SELECT ship AS name
  FROM ships.outcomes
  WHERE ship LIKE '% % %' );
```

7. Elencare nome, stazza e numero cannoni delle navi della battaglia di Guadalcanal

```
SELECT s.name, c.displacement, c.num_guns
FROM   ships.outcomes o JOIN ships.ships s ON o.ship = s.name
      JOIN ships.classes c ON c.class = s.class
WHERE  o.battle = 'Guadalcanal';
```

Esercizio 2 – Soluzioni (3/4)

8. Elencare le navi con stazza maggiore di 37000 t

```
SELECT s.name  
FROM ships.ships s NATURAL JOIN ships.classes c  
WHERE c.displacement > 37000;
```

9. Elencare le navi presenti nel DB (attenzione a navi in Outcomes non in Ships)

```
( SELECT name  
  FROM ships.ships )  
UNION  
( SELECT ship AS name  
  FROM ships.outcomes );
```

Esercizio 2 – Soluzioni (4/4)

10. Trovare le battaglie con almeno due navi dello stesso paese

```
SELECT DISTINCT o1.battle
FROM    ships.ships s1, ships.classes c1, ships.outcomes o1,
         ships.ships s2, ships.classes c2, ships.outcomes o2
WHERE   s1.class = c1.class AND s1.name = o1.ship AND
         s2.class = c2.class AND s2.name = o2.ship AND
         s1.name <> s2.name AND o1.battle = o2.battle
```

11. Trovare i paesi che avevano sia corazzate (bb) che incrociatori (bc)

```
SELECT DISTINCT c1.country
FROM    ships.classes c1, ships.classes c2
WHERE   c1.country = c2.country AND
         c1.type = 'bb' AND c2.type = 'bc';
```

Esercizio 3 – Visite mediche

paziente (id_paziente, codice_asl, provincia_asl, nome, cognome, data_nascita, provincia_nascita)

visita (id_visita, id_paziente, id_medico, data, peso, altezza, pressione_minima, pressione_massima)

medico (id_medico, nome, cognome, sesso, codice_asl, provincia_asl, indirizzo_ambulatorio)

Esprimere in SQL le seguenti query:

1. Per ciascuna visita, elencare data, cognome paziente e peso misurato
2. Trovare nome e cognome dei pazienti che hanno avuto pressione max ≥ 140
3. Elencare le date in cui la dott.ssa RUSSI ha visitato pazienti
4. Elencare nome e cognome dei pazienti visitati nell'agosto 2010
5. Elencare le visite nelle ASL di Milano: per ognuna, ritornare cognome paziente, data visita, peso e altezza misurati, ordinando per cognome e data visita
6. Elencare, per ciascun medico, i pazienti visitati
7. Ritornare nome e cognome di tutti i pazienti dell'ASL di Reggio Emilia, includendo peso e altezza misurati nelle varie visite se ci sono dati disponibili
8. Elencare nome e cognome di tutti i medici con i pazienti visitati (se ce ne sono)
9. Elencare i pazienti che non hanno effettuato visite
10. Elencare i pazienti che sono stati visitati da dottoresse

Esercizio 3 – Soluzioni (1/3)

1. Per ciascuna visita, elencare data, cognome paziente e peso misurato

```
SELECT data, cognome, peso
FROM med.visita NATURAL JOIN med.paziente;
```

2. Trovare nome e cognome dei pazienti che hanno avuto pressione max ≥ 140

```
SELECT DISTINCT nome, cognome
FROM med.paziente NATURAL JOIN med.visita
WHERE pressione_massima  $\geq 140$ ;
```

3. Elencare le date in cui la dott.ssa RUSSI ha visitato pazienti

```
SELECT DISTINCT data
FROM med.visita NATURAL JOIN med.medico
WHERE cognome = 'RUSSI';
```

4. Elencare nome e cognome dei pazienti visitati nell'agosto 2010

```
SELECT DISTINCT nome, cognome
FROM med.paziente NATURAL JOIN med.visita
WHERE data  $\geq$  '2010-08-01' AND data  $\leq$  '2010-08-31';
```

Esercizio 3 – Soluzioni (2/3)

5. Elencare le visite nelle ASL di Milano: per ognuna, ritornare cognome paziente, data visita, peso e altezza misurati, ordinando per cognome e data visita

```
SELECT cognome, data, peso, altezza
FROM med.paziente NATURAL JOIN med.visita
WHERE provincia_asl = 'MI'
ORDER BY cognome, data;
```

6. Elencare, per ciascun medico, i pazienti visitati

```
SELECT DISTINCT m.cognome AS cognome_medico, m.nome AS nome_medico,
p.cognome AS cognome_paziente, p.nome AS nome_paziente
FROM med.paziente p
JOIN med.visita v ON p.id_paziente = v.id_paziente
JOIN med.medico m ON v.id_medico = m.id_medico;
```

7. Ritornare nome e cognome di tutti i pazienti dell' ASL di Reggio Emilia, includendo peso e altezza misurati nelle varie visite se ci sono dati disponibili

```
SELECT nome, cognome, peso, altezza
FROM med.paziente NATURAL LEFT OUTER JOIN med.visita
WHERE provincia_asl = 'RE';
```

Esercizio 3 – Soluzioni (3/3)

8. Elencare nome e cognome di tutti i medici con i pazienti visitati (se ce ne sono)

```
SELECT DISTINCT m.cognome AS cognome_medico, m.nome AS nome_medico,  
p.cognome AS cognome_paziente, p.nome AS nome_paziente  
FROM med.medico m  
      LEFT OUTER JOIN med.visita v ON m.id_medico = v.id_medico  
      LEFT OUTER JOIN med.paziente p ON v.id_paziente = p.id_paziente;
```

9. Elencare i pazienti che non hanno effettuato visite

```
SELECT cognome, nome  
FROM med.paziente NATURAL LEFT OUTER JOIN med.visita  
WHERE id_visita IS NULL;
```

10. Elencare i pazienti che sono stati visitati da dottoresse

```
SELECT DISTINCT p.cognome, p.nome  
FROM med.paziente p  
      JOIN med.visita v ON p.id_paziente = v.id_paziente  
      JOIN med.medico m ON v.id_medico = m.id_medico  
WHERE m.sesso = 'F';
```

Esercizi dal libro

- Gli esercizi 1 e 2 corrispondono agli esercizi 6.1.3, 6.2.2 e 6.1.4, 6.2.3 del libro
- In aggiunta, si propongono gli esercizi 6.1.2 e 6.2.1

Esercizi dal libro – 6.1.2

Exercise 6.1.2: Write the following queries, based on our running movie database example

```
Movies(title, year, length, genre, studioName, producerC#)
StarsIn(movieTitle, movieYear, starName)
MovieStar(name, address, gender, birthdate)
MovieExec(name, address, cert#, netWorth)
Studio(name, address, presC#)
```

- a) Find Aishwara Rai's birthdate.
- b) Find the address of Film City.
- c) Find all the stars that appeared either in a movie made in 2000 or a movie with "Story" in the title.
- d) Find all the stars who either are female or live in Mumbai (have string Mumbai as a part of their address).
- e) Find all executives worth at least \$20,000,000.

Esercizi dal libro – 6.2.1

Exercise 6.2.1: Using the database schema of our running movie example

```
Movies(title, year, length, genre, studioName, producerC#)
StarsIn(movieTitle, movieYear, starName)
MovieStar(name, address, gender, birthdate)
MovieExec(name, address, cert#, netWorth)
Studio(name, address, presC#)
```

write the following queries in SQL.

- a) Who is the president of Film City?
- b) Who were the male stars in *Monsoon Wedding*?
- c) Which stars appeared in movies produced by Sony in 2005?
- ! d) Which executives are worth more than Subhash Ghai?
- ! e) Which movies are longer than *Bride and Prejudice*?